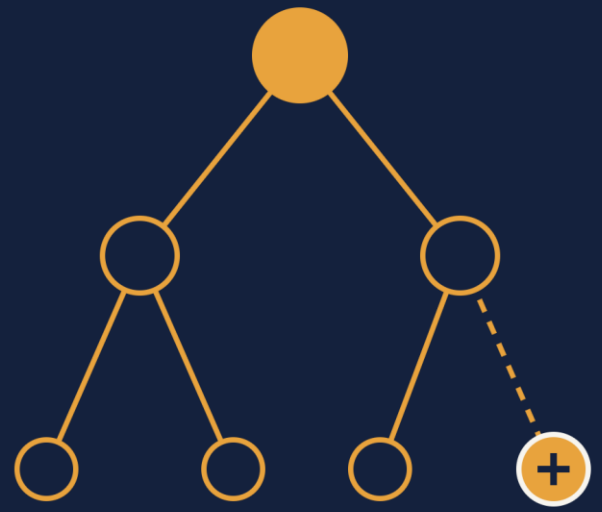
DWEC · UNIDAD 3.2

Modificando el documento

Texto, elementos y atributos: crear, insertar, cambiar y eliminar con JavaScript



ÍNDICE

Qué vamos a ver

01 **Contenido de texto**

02 **Crear e insertar elementos**

03 **Clonar y eliminar**

04 **Tipos de elementos**

05 **Atributos y clases**

Punto de partida: ya sabemos **seleccionar** elementos (UT 3.1). Ahora aprenderemos a **cambiar** la página sin recargarla.

VISIÓN GENERAL

El ciclo para modificar una página

1 **Seleccionar**
`querySelector()`

→

2 **Crear**
`createElement()`

→

3 **Configurar**
`textContent`, `className`…

→

4 **Insertar**
`append()`, `before()`…

Propiedad / método	Para qué sirve
textContent	Leer o cambiar el texto de un nodo
innerHTML	Leer o cambiar el HTML interior de un elemento
createElement() · createTextNode()	Crear nodos nuevos (aún fuera de la página)
append() · remove() · setAttribute()	Insertar, eliminar y cambiar atributos

Leer el texto: textContent

```html
<ul id="noticias">
  <li>
    <strong>Noticias:</strong> Nuevo álbum
    de <em>Ben Harper</em> a la venta.
  </li>
</ul>
```

```js
const noticia = document.querySelector('#noticias li');
console.log(noticia.textContent);
// Noticias: Nuevo álbum de Ben Harper a la venta.
```

Solo texto

Devuelve el texto de todos los descendientes y **descarta las etiquetas** , …

Ideal para leer

Casi nunca nos importa qué marcado tenía el texto: solo su contenido.

1 · CONTENIDO DE TEXTO

Cambiar el texto: textContent =

```
const noticia = document.querySelector('#noticias li');
noticia.textContent = 'Noticia: el nuevo álbum de Tool aún no ha salido.';

noticia.textContent = '<strong>Noticia:</strong> nuevo álbum';   // ¡no interpreta el HTML!
```

PRIMERA ASIGNACIÓN

- Noticia: el nuevo álbum de Tool aún no ha salido.

SEGUNDA ASIGNACIÓN

- `<strong>Noticia:</strong> nuevo álbum`

Con textContent las etiquetas se muestran **como texto literal**. Es la opción segura para escribir datos que vienen del usuario.

Leer y escribir HTML: innerHTML

```
const noticia = document.querySelector('#noticias li');

// Escribir: el marcado SÍ se interpreta
noticia.innerHTML = '<strong>Noticia:</strong> nuevo álbum de <em>Tool</em>';

// Leer: devuelve el texto CON las etiquetas
console.log(noticia.innerHTML);
// <strong>Noticia:</strong> nuevo álbum de <em>Tool</em>
```

RESULTADO EN LA PÁGINA

- **Noticia:** nuevo álbum de *Tool*

⚠ Nunca metas en innerHTML texto escrito por el usuario: podría inyectar código (XSS). Para texto, usa textContent.

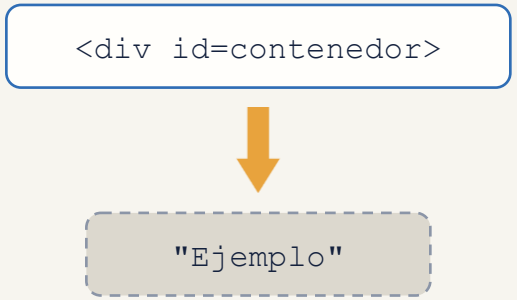
textContent, innerHTML o innerText

Propiedad	Lee	Escribe	¿Cuándo usarla?
textContent	Todo el texto, sin etiquetas	Texto literal (seguro)	Por defecto para texto
innerHTML	Texto con etiquetas	HTML que se interpreta	Fragmentos HTML de confianza
innerText	Solo el texto visible (según el CSS)	Texto literal	Casos concretos; es más lenta

Hoy **innerText** es estándar y funciona en todos los navegadores, pero calcula estilos y es más lenta: **textContent** sigue siendo la opción recomendada.

Crear nodos de texto: createTextNode()

```
const contenedor = document.getElementById('contenedor');
const nodoTexto = document.createTextNode('Ejemplo');
contenedor.appendChild(nodoTexto);    // o contenedor.append(nodoTexto)
```



Dos pasos

Crear el nodo no lo muestra: hay que **añadirlo** al árbol DOM.

Solo en document

createTextNode(), createElement() y createAttribute() solo existen en el objeto document.

Crear un elemento en 3 pasos

```
// 1. Crear el elemento <div>
const mensaje = document.createElement('div');

// 2. Configurarlo: clase y contenido
mensaje.className = 'alerta';
mensaje.innerHTML = '<strong>¡Hola!</strong> Has leído un mensaje importante.';

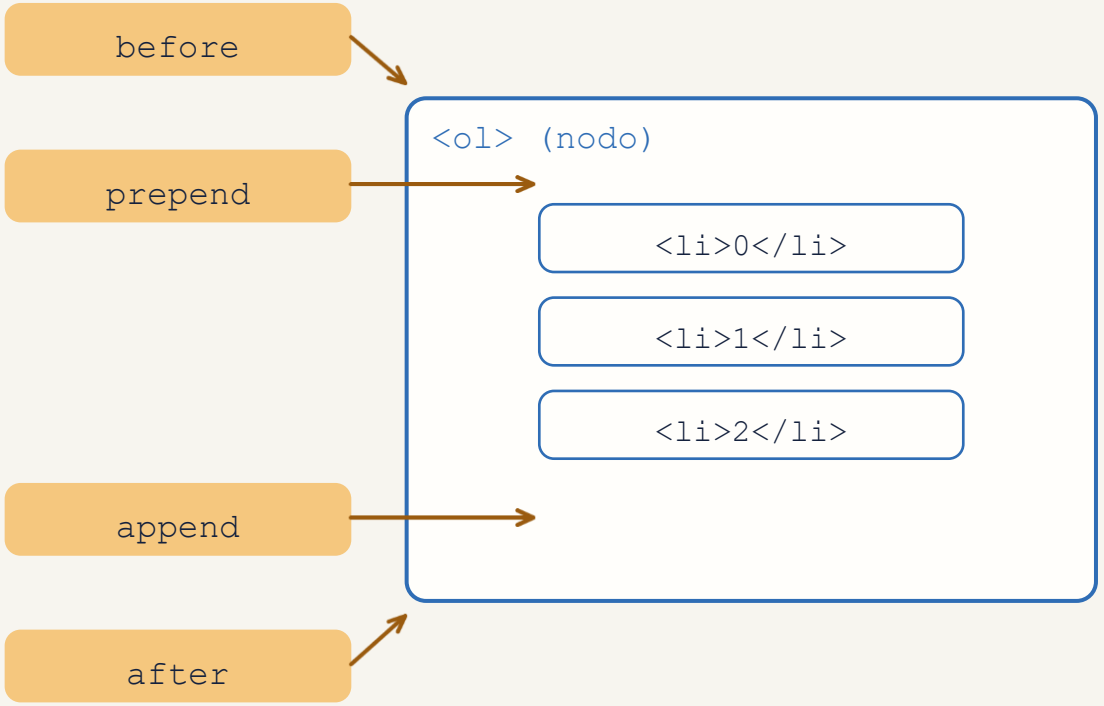
// 3. Insertarlo en la página
document.body.append(mensaje);
```

RESULTADO (CON LA CLASE CSS .ALERTA)

¡Hola! Has leído un mensaje importante.

2 · CREAR E INSERTAR

Métodos de inserción modernos



Método	Inserta…
nodo.append(…)	Al final, dentro del nodo
nodo.prepend(…)	Al principio, dentro del nodo
nodo.before(…)	Antes del nodo, fuera
nodo.after(…)	Después del nodo, fuera
nodo.replaceWith(…)	Sustituye el nodo

Aceptan **varios** nodos o cadenas a la vez: append(a, b, 'texto').

2 · CREAR E INSERTAR

Inserción en acción

```
const lista = document.getElementById('lista');

lista.before('antes');   // texto antes de <ol>
lista.after('después');  // texto después de <ol>

const primero = document.createElement('li');
primero.textContent = 'prepend';
lista.prepend(primero);  // al principio

const ultimo = document.createElement('li');
ultimo.textContent = 'append';
lista.append(ultimo);    // al final
```

RESULTADO EN LA PÁGINA

antes

1. **prepend**
2. 0
3. 1
4. 2
5. **append**

después

Las cadenas se insertan como texto

```
const caja = document.getElementById('caja');
caja.before('<p>Hola</p>', document.createElement('hr'));
```

RESULTADO EN LA PÁGINA

Hola

[div#caja]

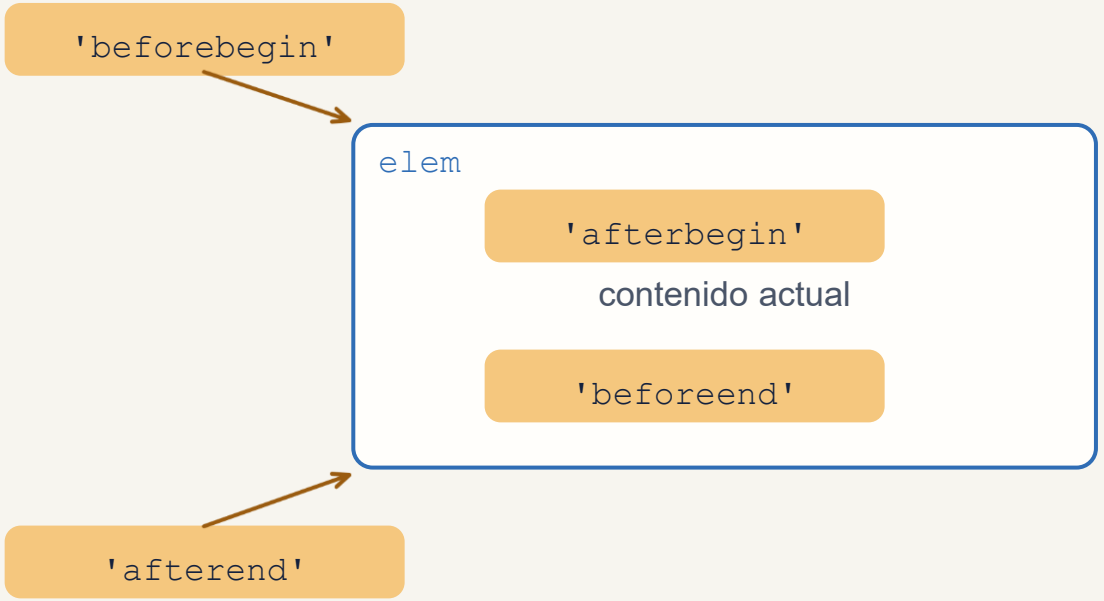
Como textContent

Los caracteres < y > se escapan: la etiqueta se ve, no se interpreta.

¿Y si quiero HTML?

Usa **insertAdjacentHTML()**: lo vemos a continuación.

Insertar HTML: insertAdjacentHTML()




```
const caja = document.getElementById('caja');
caja.insertAdjacentHTML('beforebegin',
'<p>Hola</p>');
caja.insertAdjacentHTML('afterend', '<p>Adiós</p>');
```

Mismas posiciones que before / prepend / append / after, pero el texto **se interpreta como HTML**.

Hermanos: **insertAdjacentText(pos, texto)** e **insertAdjacentElement(pos, elemento)**. En la práctica se usa casi siempre la versión HTML.

El mismo mensaje con plantillas de texto

```
document.body.insertAdjacentHTML('afterbegin', `
  <div class="alerta">
    <strong>¡Hola!</strong> Has leído un mensaje importante.
  </div>
`);
```

Plantillas ES6

Las comillas invertidas (`) permiten escribir HTML en varias líneas e incluir variables con ${…}.

Más corto

Sin createElement ni className: ideal para fragmentos de HTML fijos y de confianza.

Cuidado

Si interpolas datos del usuario en la plantilla, vuelves a tener riesgo de XSS.

Clonar nodos: cloneNode()

```
const mensaje = document.getElementById('mensaje');

const copia = mensaje.cloneNode(true);    // copia profunda
copia.id = 'mensaje-2';                    // ¡los id deben ser
únicos!
copia.querySelector('strong').textContent = '¡Adiós!';

mensaje.after(copia);
```

RESULTADO EN LA PÁGINA

¡Hola! Mensaje importante.

¡Adiós! Mensaje importante.

cloneNode(true)

Copia el elemento con todos sus atributos y **todos sus descendientes**.

cloneNode(false)

Copia solo el elemento y sus atributos, **sin hijos**.

Insertar muchos nodos a la vez

Con DocumentFragment

```
const crearElementos = () => {
  const fragmento = new DocumentFragment();
  for (let i = 1; i <= 3; i++) {
    const li = document.createElement('li');
    li.append(i);
    fragmento.append(li);
  }
  return fragmento;
};
lista.append(crearElementos());
```

Con un array y spread (ES6)

```
const crearElementos = () => {
  const elementos = [];
  for (let i = 1; i <= 3; i++) {
    const li = document.createElement('li');
    li.append(i);
    elementos.push(li);
  }
  return elementos;
};
lista.append(...crearElementos());
```

Al insertar un fragmento se inserta **su contenido**, no el fragmento. Con append y el operador **...** rara vez hace falta.

Eliminar nodos

Clásico: removeChild()

```
const cuerpo = document.querySelector('tbody');
// Vaciar la tabla fila a fila
while (cuerpo.firstChild !== null) {
  cuerpo.removeChild(cuerpo.firstChild);
}
```

Moderno: remove() y replaceChildren()

```
// Eliminar un elemento directamente
document.querySelector('#aviso').remove();

// Vaciar un elemento de golpe
cuerpo.replaceChildren();
```

removeChild(hijo)

Se llama sobre el **padre** y devuelve el nodo eliminado.

elem.remove()

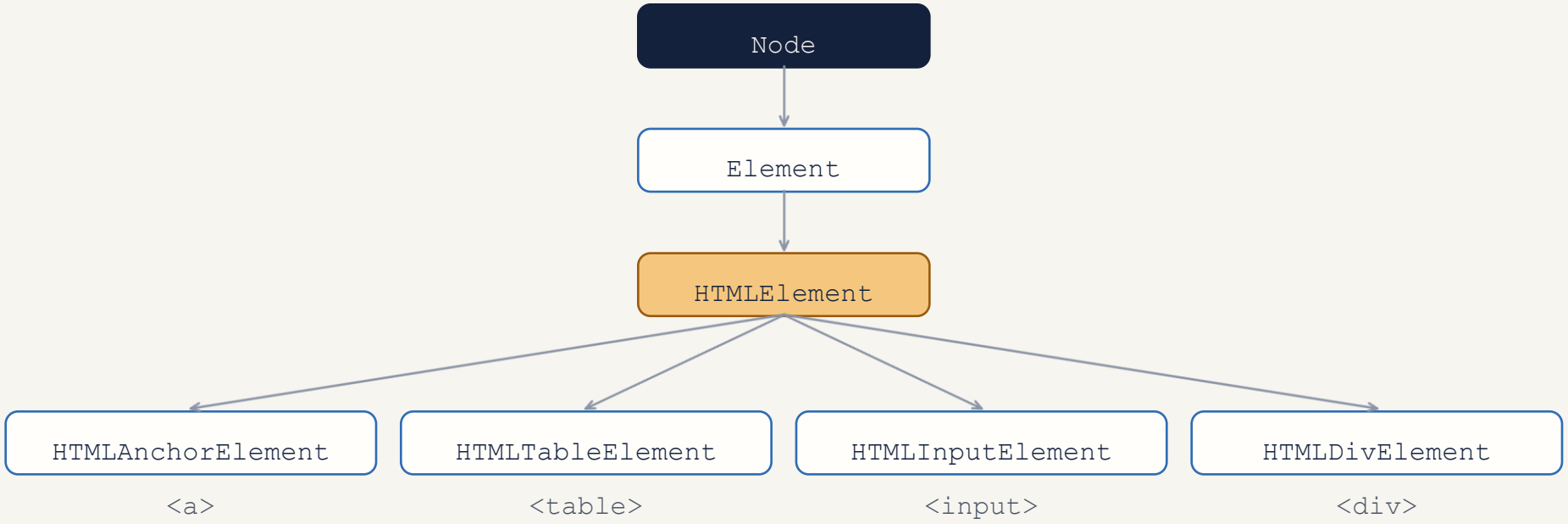
Se llama sobre el propio nodo: no necesitas conocer al padre.

Métodos clásicos y modernos

Clásico (DOM Level 1)	Moderno	Diferencia
padre.appendChild(nodo)	padre.append(…)	Varios nodos y cadenas
padre.insertBefore(nuevo, ref)	ref.before(…)	No necesita al padre
padre.replaceChild(nuevo, viejo)	viejo.replaceWith(…)	Más legible
padre.removeChild(nodo)	nodo.remove()	No necesita al padre

Los clásicos siguen funcionando y aparecen en mucho código existente: conviene reconocerlos, pero en código nuevo usa los modernos.

Cada etiqueta tiene su propio tipo



Todos heredan de **HTMLElement**. Las etiquetas sin tipo propio (, …) son directamente HTMLElement. La lista completa está en la Tabla 2 del tema.

Ejemplo: HTMLTableElement

Propiedades propias

```
const tabla = document.querySelector('table');
console.log(tabla instanceof HTMLTableElement); //
true
console.log(tabla.caption);    // null
console.log(tabla.tHead);      // <thead>
console.log(tabla.tBodies);    // [tbody]
console.log(tabla.rows);       // [tr, tr, …]
```

Métodos propios

```
const fila = tabla.insertRow(1);

const nombre = fila.insertCell(0);
nombre.textContent = 'Bob';
const apellido = fila.insertCell(1);
apellido.textContent = 'Doe';
const email = fila.insertCell(2);
email.textContent = 'bob.doe@ejemplo.es';
```

insertRow() e insertCell() crean filas y celdas sin usar createElement. Ojo: la propiedad es **tHead** (camelCase) y la etiqueta, <thead>.

Trabajar con atributos

```
<a id="inicio" class="enlace" href="index.html">Inicio</a>
```

`getAttribute(nombre)`

Leer el valor

`setAttribute(nombre, valor)`

Cambiar o añadir

`removeAttribute(nombre)`

Eliminar

`createAttribute() + setAttributeNode()`

Crear un nodo atributo

getAttribute() o propiedad

Con getAttribute()

```
const enlace = document.getElementById('inicio');
enlace.getAttribute('id');      // 'inicio'
enlace.getAttribute('class');   // 'enlace'
enlace.getAttribute('href');    // 'index.html'
```

Con propiedades

```
enlace.id;          // 'inicio'
enlace.className;   // 'enlace'    ← no es .class
enlace.href;        // 'http://…/index.html'
```



Casi todos los atributos existen también como propiedad. Excepción: **class** → **className** (class es palabra reservada). Y **href** como propiedad devuelve la URL completa.

Cuando atributo y propiedad no coinciden

```
<button id="crear" style="background-color: green" onclick="crearContacto()">Crear</button>
```

Acceso	getAttribute()	Propiedad
style	'background-color: green' (cadena)	Objeto CSSStyleDeclaration
onclick	'crearContacto()' (cadena)	Una función
href	'index.html' (valor literal)	URL absoluta

La propiedad **style** permite leer y escribir cada propiedad CSS; **onclick** permite asignar funciones como manejadores de eventos (lo veremos en la unidad de eventos).

Cambiar, añadir y eliminar atributos

setAttribute()

```
enlace.setAttribute('class', 'enlace activo');
enlace.setAttribute('href', 'nuevo.html');
enlace.setAttribute('target', '_blank');  // nuevo
```

Mediante propiedades

```
enlace.className = 'enlace activo resaltado';
enlace.href = 'otro.html';
enlace.target = '_self';
```

removeAttribute()

```
enlace.removeAttribute('class');
enlace.removeAttribute('href');
enlace.getAttribute('href');   // null
```

createAttribute() (poco usado)

```
const atributo = document.createAttribute('target');
atributo.value = '_blank';
enlace.setAttributeNode(atributo);
```

Gestionar clases CSS: classList

`add('activo')`	`remove('activo')`	`toggle('activo')`	`contains('activo')`
Añade la clase	Quita la clase	La quita si está, la pone si no	true / false

```
const enlace = document.getElementById('inicio');   // class="enlace"
enlace.classList.add('activo');         // "enlace activo"
enlace.classList.remove('activo');      // "enlace"
enlace.classList.toggle('activo');      // "enlace activo"
enlace.classList.contains('enlace');    // true

const hayResultados = 5 > 0;
enlace.classList.toggle('activo', hayResultados);   // fuerza añadir si es true
```

Mucho más cómodo que manipular la cadena de **className** a mano.

BUENAS PRÁCTICAS

Organiza el código DOM en módulos ES6

utilidadesDom.js

```
export const crearElemento = (etiqueta, texto, clases
= []) => {
  const elemento = document.createElement(etiqueta);
  elemento.textContent = texto;
  elemento.classList.add(...clases);
  return elemento;
};

export const vaciar = (elemento) =>
elemento.replaceChildren();
```

main.js

```
import { crearElemento, vaciar }
from './utilidadesDom.js';

const lista = document.querySelector('#tareas');
vaciar(lista);
lista.append(
  crearElemento('li', 'Repasar el DOM',
['pendiente']),
  crearElemento('li', 'Hacer la práctica')
);
```

```
<script type="module" src="main.js"></script>
```

RESUMEN

¿Qué uso en cada caso?

Quiero…	Uso
Leer o cambiar texto	textContent
Insertar un fragmento HTML de confianza	innerHTML · insertAdjacentHTML()
Crear un elemento nuevo	createElement() + append() / before()…
Copiar un elemento	cloneNode(true)
Eliminar un elemento	elem.remove()
Leer / cambiar atributos	getAttribute() · setAttribute() · propiedades
Añadir o quitar clases	classList.add / remove / toggle